Problema

El Club Deportivo Séneca (CDS) necesita un sistema de analítica deportiva que apoye su funcionamiento. El club cuenta con dos frentes de operación: el deportivo y el comercial. El primero gira en torno a la formación y competición de sus socios. Para esto, el sistema debe facilitar la gestión ordenada de las instalaciones y los equipos pertenecientes al club, incluyendo el monitoreo de entrenamientos, partidos, competencias y estadísticas pertinentes. El otro frente del club atiende las necesidades de la tienda deportiva y la cafetería dispuesta para los socios del club. El sistema debe facilitar el control riguroso del inventario y las ventas. Al sistema pueden acceder tres tipos de usuario: socios (jugadores), empleados del club (entrenadores y fisioterapeutas) y un administrador encargado.

Objetivos del sistema

1. Controlar quién juega, en qué equipo, bajo qué reglas y con qué resultados.
2. Hacer cumplir automáticamente las reglas de negocio (cupos, edades, suspensiones, niveles, reservas, turnos de empleados, bebidas calientes).
3. Llevar el control de inventario y ventas con sus impuestos y puntos de fidelidad.

Requerimientos Funcionales

A continuación, se desglosan y agrupan los requerimientos funcionales del sistema según el tipo de usuario que le corresponde. En ocasiones hay dos o más tipos de usuario que comparten requerimientos funcionales. Tal es el caso del RF16 que aplica de igual manera tanto para Entrenadores como para Fisioterapeutas y de los RF17-21 que aplican para todo tipo de usuario (administrador, socios, empleados).

Administrador

RF1: Permitir al administrador la creación de equipos por deporte y edad (Sub-13, Sub-15, Sub-17, Mayores)
RF2: Permitir al administrador programar partidos internos entre equipos del club. Estos deben ser de la misma categoría y deporte y se debe validar el cumplimiento del número mínimo de jugadores habilitados por equipo (7 en fútbol, 5 en baloncesto, 6 en voleibol) para programarse. (Nota: El número de jugadores habilitados es imposible validarlo mediante el sistema a la hora de jugarse el partido, por eso solo se válida para la programación de un partido. Por ejemplo, si un equipo de fútbol tiene 15 jugadores registrados, pero 9 están suspendidos para el próximo partido entonces el sistema rechaza la programación del partido).
RF3: Permitir al administrador registrar equipos en competencias externas (ASCUN y Torneo de los Cerros) verificando que se cumplan el mínimo de nómina de jugadores habilitados y la categoría permitida.

RF4: Permitir al administrador consultar tablas de estadísticas individuales acumuladas por torneo como la del goleador del torneo.
RF5: Permitir al administrador ver en detalle el estado del inventario de instalaciones y artículos, así como el historial completo de reservas, partidos y sesiones de entrenamiento.
RF6: Permitir al administrador dar de baja equipos y jugadores, y reabastecer las tiendas.
RF7: Permitir al administrador generar reportes de ventas discriminados periodo (diario, semanal, mensual), así como reportes deportivos (asistencia a entrenamientos, resultados de partidos, y rankings).

Socios

RF8: Permitir a los socios su inscripción a un equipo del club validando que existe el cupo y que el jugador cumple con el requisito de edad de la categoría.
RF9: Permitir a los socios inscribirse a deportes individuales asignándoles por defecto el nivel de Principiante.
RF10: Permitir a los socios registrar, para cada partido de deporte individual, el resultado y las estadísticas propias (en tenis y pádel, sets y games ganados, aces y errores no forzados; en tenis de mesa, sets ganados y puntos totales). Se actualiza el puntaje de ranking individual de los socios involucrados de manera automática.
RF11: Permitir a los socios programar partidos de deportes individuales con otros socios del mismo nivel o niveles adyacentes. En caso de que uno de los jugadores sea entrenador a este se le permite programar el partido también.
RF12: Permitir a los socios la reserva de canchas de tenis, pádel o tenis de mesa por bloques horarios. Para esto el socio debe especificar cuántos jugadores participarán y la modalidad. El sistema debe validar la capacidad y el cupo de la instalación a reservar.

Entrenadores

RF13: Permitir a los entrenadores registrar el marcador y las estadísticas individuales de los equipos del partido de conjunto. Al registrar tarjetas se actualiza la suspensión automáticamente.
RF14: Permitir a los entrenadores programar entrenamientos. Para cada sesión se debe especificar fecha, hora, duración, entrenador responsable, instalación utilizada y la lista de jugadores convocados. Los convocados deben pertenecer al equipo o a la disciplina de la sesión.
RF15: Permitir a los entrenadores registrar, para cada sesión de entrenamiento, la asistencia (asistió, faltó, llegó tarde) y evaluaciones de desempeño opcional de los deportistas (nota de 1 a 5 y comentario).

Empleados

RF16: Permitir a los empleados solicitar un cambio de turno. El sistema lo aprueba automáticamente si el turno que el empleado deja sigue cumpliendo el mínimo de 2 entrenadores y 1 fisioterapeuta, y lo rechaza en caso contrario.

Usuarios

RF17: Permitir a los usuarios la compra de artículos en la tienda deportiva. El sistema debe registrar la compra aplicando el 19% al IVA y los descuentos correspondientes (15% para empleados y 8% para socios) al presentarse un código de descuento válido.
RF18: Permitir a los usuarios la compra de bebidas y snacks de la cafetería. El sistema debe registrar la compra aplicando el 8% de impoconsumo y sugerencia de propina del 10%.
RF19: Informar a los usuarios sobre los alérgenos contenidos por los snacks que desea comprar antes de que se realice la compra.
RF20: Permitir a los usuarios (socios, empelados y administrador) acumular puntos de fidelidad por cada compra en cualquier tienda (2% del valor total) de empleados y socios.
RF21: Permitir a los usuarios (socios, empleados y administrador) la redención de puntos de fidelidad como descuento de una compra (1 punto = 1 peso) a empleados y socios.

Requerimientos No Funcionales

RNF1: Las reglas que varían por deporte (puntos por victoria, empate y derrota, si permite empate, mínimo de jugadores, posiciones) se guardan como datos en Deporte. Agregar o ajustar un deporte no debe resultar en un cambio en la lógica interna de partidos ni de competencias.
RNF2: Toda la información se guarda en archivos dentro de una carpeta distinta a la del código fuente. Al reiniciar la aplicación se restaura el estado completo, y cada operación que modifica datos queda guardada al terminar.
RNF3: Todo usuario se autentica con login y password. Cada operación solo está disponible para el rol autorizado (por ejemplo, solo el Administrador puede acceder reportes de ventas, pero no puede reservar ni inscribirse a equipos).
RNF4: Las operaciones que violen una regla de negocio se rechazan completamente y no deja el sistema en un estado parcial (por ejemplo, una venta rechazada no descuenta inventario ni suma puntos).
RNF5: Cada operación rechazada dado el incumplimiento de una regla de negocio devuelve un mensaje que indica cuál regla se incumplió.

Restricciones

1. La aplicación debe estar hecha en Java
2. Toda la información debe ser persistente: se supone que solo la aplicación va a escribir y leer de esa carpeta, y ningún usuario malicioso va a modificar los archivos que ahí se encuentren sin utilizar la aplicación.
3. La carpeta con la información debe encontrarse en una carpeta separada a la del código fuente.
4. Todos los usuarios del sistema deben tener un login y un password.

No Objetivos

1. Gestionar los pagos y las transacciones de la tienda deportiva y cafetería. El sistema solo registra y valida ventas.
2. Llevar registro del cumplimiento del pago de mensualidades
3. Llevar registro y gestión del movimiento de artículos entre las distintas ubicaciones de inventario
4. Acumular puntos de rivales externos y actualizar tablas de posiciones/clasificaciones de competencias externas
5. Registrar estadísticas de jugadores pertenecientes a equipos rivales externos y actualizar clasificaciones de estadísticas individuales de competencias externas.
6. Verificar y validar que los jugadores alineados en un partido interno o de competencia externa estén habilitados y no tengan suspensiones activas.
7. Verificar que se presente el número mínimo de jugadores a un partido.
8. Restringir el uso de instalaciones techadas a socios con bebidas calientes. Es una regla de negocio que un sistema no puede garantizar.
9. Verificar que una instalación esté libre en el horario de un entrenamiento, u otras reglas implícitas no mencionadas en el enunciado.
10. Registrar la evolución histórica del ranking. Solo se lleva registro del ranking actual.
11. Interfaz gráfica: el sistema se usa únicamente por consola.
12. Aprobar sugerencias de nuevos artículos o snacks hechas por empleados.
13. Revisión manual de las solicitudes de cambio de turno por parte del administrador: el sistema las aprueba o rechaza automáticamente según el personal mínimo.
14. Llevar el conteo de tarjetas y suspensiones de cada competencia de forma separada del conteo interno.
15. Relacionar a los entrenadores con equipos. Cualquier entrenador registra marcadores y estadísticas.
16. Modificar la propina sugerida. Esta queda fija en 10%.

La existencia de algunos no objetivo se debe a las limitaciones que presenta el dominio del problema (4, 5). Otros se deben a las limitaciones intrínsecas de un programa (6, 7, 8). En otras ocasiones consideramos que el enunciado presentaba ambigüedades o no era lo suficientemente específico (1, 9). No obstante, también hubo decisiones estratégicas donde, dado el tiempo limitado de la entrega se decidió priorizar la implementación de ciertas funcionalidades sobre otras que también eran mencionadas en el enunciado. Si fueron omitidas y asumidas como no objetivos, se debe a que consideramos que estas no eran vitales para el funcionamiento general del sistema o por falta de tiempo (2, 3, 10, 12, 13, 14, 15, 16).

Componentes del Sistema

1. Usuarios y acceso: contiene la jerarquía de usuarios, conoce los login y contraseñas de cada usuario y los autentica

2. Lógica Deportiva
3. Lógica Comercial
4. Persistencia
5. Interfaz (Consola)

Diagrama de Componentes

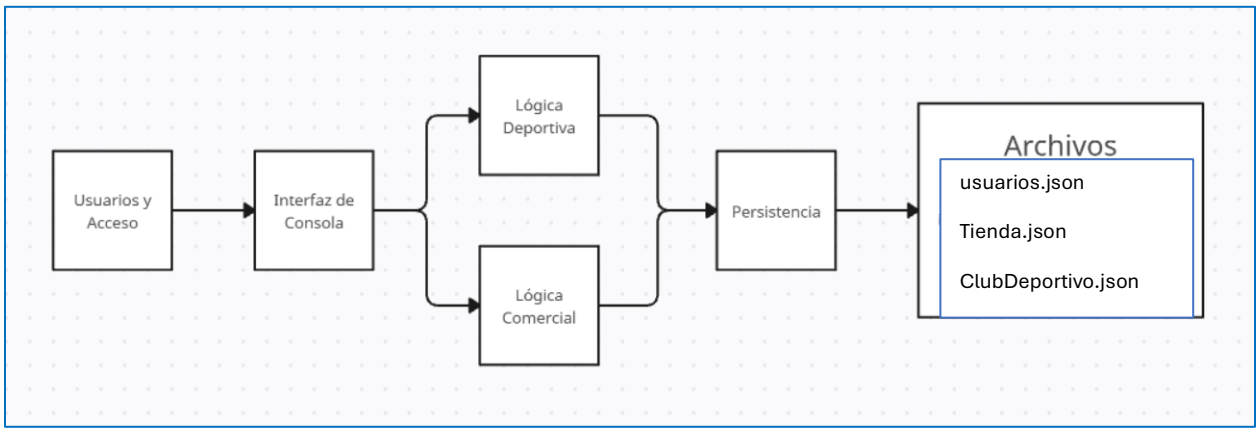
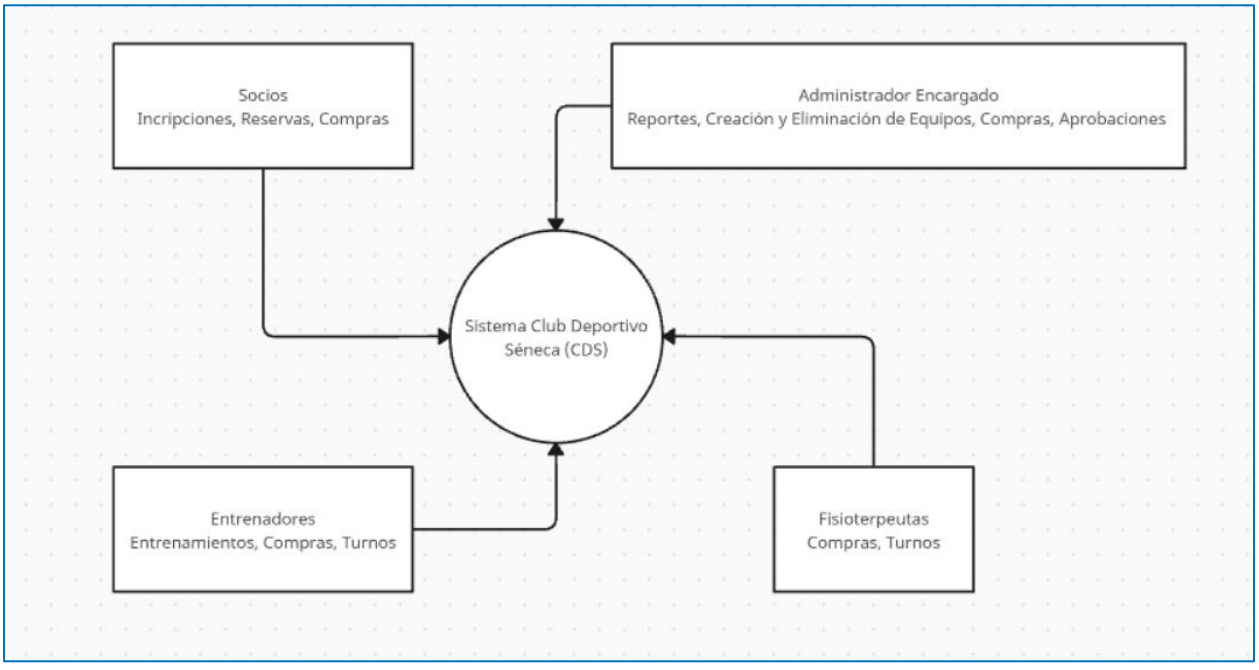


Diagrama de contexto



Candidatos y Estereotipos

Se agruparon los candidatos por componentes del sistema para llevar un mayor control y orden de las adiciones y cambios realizados durante el proceso iterativo del diseño. En ocasiones se

escogió agrupar candidatos en una misma celda para evitar la repetición de la descripción; si bien son candidatos distintos, su descripción es casi idéntica.

Usuarios y Acceso

Candidato	Descripción	Estereotipo
Usuario (abstracta)	Guarda login, password, nombre y puntos fidelidad.	Information Holder
Administrador, Fisioterapeuta	Guardan los datos particulares de cada rol	Information Holder
Socio, Empleado, Entrenador	Agrupan lo que le pertenece a cada uno: el socio sus inscripciones, reservas, asistencias y estadísticas. El empleado sus turnos. El entrenador sus sesiones. El socio decide si puede inscribirse a un equipo o a una disciplina	Structurer
Autenticador	Verifica que un login esté disponible y autentica login y contraseña con respecto a los usuarios registrados	Service Provider

Lógica Deportiva

Candidato	Descripción	Estereotipo
Deporte	Guarda las reglas del deporte	Information Holder
Equipo	Conoce sus inscripciones y su cupo	Structurer
Inscripción (abstracta), InscripciónEquipo, InscripciónIndividual, InscripciónCompetencia	Vínculo entre un socio y un equipo, un socio y un deporte, o un equipo y una competencia, con sus datos propios	Information Holder
Competencia	Agrupa los equipos inscritos y sus partidos	Structurer
Partido (abstracta), PartidoInterno, PartidoCompetencia, PartidoIndividual	Agrupa los participantes del partido y su marcador y sus estadisticas	Structurer
Estadistica (abstracta) y sus subclases por deporte	Contiene las estadísticas de un jugador en un partido	Information Holder

Reserva	Contiene los datos de la reserva	Information Holder
Instalacion	Conoce sus horarios y decide si un horario existe o se puede reservar	Structurer
SesionEntrenamiento	Agrupa a los jugadores convocados y su asistencia	Structurer
AsistenciaEvaluacion, SolicitudCambioTurno	Contiene datos de asistencia, de una solicitud de cambio de turno	Information Holder
Turno	Agrupa al personal de su hora y decide si puede perder a alguien.	Structurer
ClubDeportivo	Conoce usuarios, deportes, equipos, competencias, inscripciones, instalaciones, partidos, sesiones, turnos y estadísticas. Además, ejecuta las operaciones del club y reparte las solicitudes	Coordinator

Lógica Comercial

Candidato	Descripción	Estereotipo
Producto (abstracta), ArticuloTienda, ProductoCafeteria	Contiene los datos del producto e inventario	Information Holder
Venta	Agrupa los ítems y calcula subtotal, descuento, impuestos y total (las tasas son constantes de la clase).	Structurer
ItemVenta	Una línea de la venta. Relaciona cuantas unidades de un producto se asocian a una misma venta.	Information Holder
CalculadoraDescuento	Guarda los descuentos (15% de empleado y 8% por código) y la lista de códigos válidos, y calcula el descuento que corresponde a un usuario	Service Provider
Tienda	Conoce productos y ventas, decide si una venta se puede realizar.	Coordinator

Transversales

Candidato	Descripción	Estereotipo
Consola	Autentica al usuario, lo dirige al menú de su rol (administrador, socio, entrenador o fisioterapeuta). También lee y muestra los datos de cada operación y guarda después de cada operación.	Controller, Interfacer
PersistenciaUsuarios, PersistenciaClubDeportivo, PersistenciaTienda	Leen y escriben a los archivos JSON y reconstruyen los objetos	Interfacer

Descartados

- **Inventario:** es una consulta sobre cantidadInventario.
- **Carrito:** es una Venta en progreso.
- **Jugador:** es un Socio con una InscripcionEquipo.
- **Nómina y Alineación:** se derivan de InscripcionCompetencia y de las estadísticas.
- **Ranking:** es el atributo puntajeRanking.
- **Factura o Recibo:** son Venta e ItemVenta.
- **Reporte:** se calcula, no se guarda.
- **Tabla de posiciones, Mensualidad, Ubicación de inventario y Equipo externo:** son no-objetivos.
- **Fase de competencia:** ningún RF la requiere.
- **Operacion**: Se usan nombre de operación como String
- **Clases de Control**: Las reglas que pueden validar son manejadas internamente por las entidades y/o por las clases coordinadoras.

Responsabilidades

Usuarios y Acceso

	Responsabilidad	Tipo	Asignado a
1	Saber id, login, contraseña, nombre, apellido, puntos de fidelidad y ventas del usuario. Además, el socio conoce edad, el empleado, sus turnos y su código de descuento, el entrenador, sus sesiones	Saber	Usuario, Socio, Empleado, Entrenador

2	Sumar y restar puntos de fidelidad, y agregar ventas a su historial	Hacer	Usuario
3	Agregar inscripciones, reservas, asistencias y estadísticas al socio; turnos al empleado; sesiones al entrenador	Hacer	Socio, Empleado, Entrenador
4	Decidir si un socio puede inscribirse a un equipo o a una disciplina (cupo, posición, camiseta, un equipo por deporte), consultando al equipo	Decidir	Socio
6	Verificar que un login esté disponible, y validar las credenciales de acceso	Hacer	Autentificador

Lógica Deportiva

	Responsabilidad	Tipo	Asignado a
7	Saber las reglas de cada deporte (empates, mínimo de jugadores, posiciones, puntos por resultado)	Saber	Deporte
8	Saber los socios, inscripciones y cupo de un equipo	Saber	Equipo
9	Decidir si un equipo tiene cupo y si una posición y una camiseta están disponibles	Decidir	Equipo
10	Decidir si un equipo cumple el mínimo de jugadores habilitados y si un socio puede jugar	Decidir	Equipo
11	Decidir si un equipo puede inscribirse en una competencia, jugar un partido de competencia o programar un partido interno contra otro equipo (misma categoría y deporte)	Decidir	Equipo
12	Agregar un socio y una inscripción de competencia al equipo	Hacer	Equipo
13	Acumular tarjetas amarillas y marcarse suspendida	Hacer	InscripcionEquipo
14	Actualizar su nivel y su puntaje de ranking	Hacer	InscripcionIndividual

15	Saber los partidos e inscripciones de una competencia, agregarlos y cambiar su fase	Hacer	Competencia
16	Agregar una estadística a un partido	Hacer	Partido
17	Saber sus horarios y decidir si un horario existe y se puede reservar	Decidir	Instalacion
18	Decidir si una reserva está activa en un momento dado	Decidir	Reserva
19	Agregar el equipo, los socios convocados y la asistencia y evaluación de cada uno a la sesión	Hacer	SesionEntrenamiento
20	Rechazar una nota fuera del rango de 1 a 5	Decidir	AsistenciaEvaluacion
21	Marcarse como aprobada o no	Hacer	SolicitudCambioTurno
22	Decidir si el turno cumple el mínimo de 2 entrenadores y 1 fisioterapeuta	Decidir	Turno
23	Agregar y quitar un empleado de un turno	Hacer	Turno
24	Ejecutar el flujo de cada operación deportiva (decidir, crear o modificar, guardar en la colección, persistir)	Hacer	ClubDeportivo
25	Crear y dar de baja (marcar inactivos) equipos y jugadores, y dar de alta usuarios	Hacer	ClubDeportivo
26	Decidir si dos socios, o un socio y un entrenador, pueden jugar un partido individual (niveles iguales o adyacentes)	Decidir	ClubDeportivo
27	Decidir si los convocados de una sesión pertenecen al equipo o a la disciplina	Decidir	ClubDeportivo
28	Registrar el marcador final y marcar el partido finalizado, rechazando empates si el deporte no los permite	Hacer	ClubDeportivo

29	Registrar las estadísticas de un partido y actualizar suspensiones (según el tipo de partido) y ranking	Hacer	ClubDeportivo
30	Calcular el goleador de una competencia	Hacer	ClubDeportivo
31	Realizar una solicitud de cambio de turno: consultar al turno actual, asignar al empleado al turno nuevo y retirarlo del actual	Hacer	ClubDeportivo

Lógica Comercial

	Responsabilidad	Tipo	Asignado a
32	Saber nombre, precio base, inventario y datos propios del producto (categoría, bebida caliente, alérgenos)	Saber	Producto y subclases
33	Decidir si una venta se puede realizar (unidades suficientes, puntos suficientes, bebida caliente con reserva activa)	Decidir	Tienda
34	Calcular el subtotal de una línea y descontar su cantidad del inventario	Hacer	ItemVenta
35	Armar la lista de ítems de la compra (el carrito)	Hacer	Consola
36	Calcular subtotal, descuento, impuesto, propina sugerida, total y puntos ganados según su tipo (tienda deportiva o cafetería), y restar los puntos redimidos	Hacer	Venta
37	Saber las tasas fijas (IVA, impuesto al consumo, propina sugerida y puntos de fidelidad)	Saber	Venta
38	Decidir si un código de descuento es válido y qué porcentaje otorga (15% el propio, 8% el de otro empleado)	Decidir	CalculadoraDescuento
39	Registrar los códigos de descuento válidos	Hacer	CalculadoraDescuento

40	Realizar una venta, con o sin puntos: decidir si se puede, descontar inventario, mover los puntos del usuario, registrar la venta y guardar	Hacer	Tienda
41	Consultar el inventario, informar los alérgenos de un producto, y calcular reportes de ventas	Hacer	Tienda

Transversales

	Responsabilidad	Tipo	Asignado a
42	Leer los archivos al abrir y reconstruir los objetos y sus relaciones	Hacer	PersistenciaUsuarios, PersistenciaClubDeportivo, PersistenciaTienda
43	Escribir en archivos los datos modificados	Hacer	PersistenciaUsuariosJ, PersistenciaClubDeportivo, PersistenciaTienda
44	Despachar al menú según el tipo de usuario autenticado	Hacer	Consola
45	Mostrar el menú del rol, decidir qué pedir y llamar a la coordinadora	Hacer	Consola
46	Pedir y mostrar datos por consola (cadenas, números, confirmaciones y menús)	Hacer	Consola
47	Mostrar mensajes de error y los alérgenos de un snack antes de confirmar la compra	Hacer	Consola

Decisiones de Diseño

El sistema se organiza en cinco componentes: usuarios y acceso, lógica deportiva, lógica comercial, persistencia e interfaz. La partición sigue los dos frentes del enunciado, el deportivo y el comercial, que casi no se tocan. El único punto de contacto es el usuario, que participa en la operación deportiva y también compra. Separarlos permite que cada uno cambie por sus propias razones: la tienda por productos, ventas y descuentos, y el club por deportes, equipos y competencias.

Herencia. Se usa donde hay una relación "es un" y los tipos comparten datos y comportamiento:

- Usuario guarda una sola vez los datos de acceso y los puntos de fidelidad, y de él heredan Socio, Empleado y Administrador. Entrenador y Fisioterapeuta heredan de Empleado, porque comparten los turnos.
- Producto reúne nombre, precio e inventario, y de él heredan ArticuloTienda y ProductoCafeteria, que agregan sus datos propios.
- Partido reúne marcador, estado, deporte y estadísticas, y de él heredan PartidoInterno, PartidoCompetencia y PartidoIndividual, que se diferencian en quiénes participan.
- Estadistica reúne el jugador y el partido, y cada deporte aporta las suyas, porque las métricas cambian de un deporte a otro.

Composición en lugar de herencia. Donde la diferencia es solo de valores, no hay subclases: Deporte es una sola clase parametrizada con sus reglas. De esta manera, agregar o ajustar un deporte no obliga a cambiar la lógica de partidos ni de competencias. Tampoco hay una superclase para las inscripciones porque cada una vincula elementos distintos (socio y equipo, socio y disciplina, equipo y competencia) y no comparten atributos que justifiquen una jerarquía.

Contención. Cada objeto es dueño de sus partes: Venta contiene sus ItemVenta, Partido sus estadísticas, SesionEntrenamiento sus asistencias, Instalacion sus horarios y reservas, Competencia sus inscripciones y partidos, y Socio sus inscripciones. Así la información queda junto a quien la usa.

Coordinadoras. ClubDeportivo y Tienda son el único punto de entrada de sus componentes y las dueñas de las colecciones principales. Reciben la solicitud de la consola, se apoyan en las entidades para decidir, crean o modifican y guardan.

El control se reparte siguiendo el principio de Experto en Información. Esto quiere decir que cada regla la decide la clase que tiene los datos necesarios. No hay clases de control separadas. Estas se evaluaron y se descartaron, porque exigían pasarles los objetos por parámetro para que preguntaran lo que los propios objetos ya saben.

Algunos ejemplos:

- Equipo decide si tiene cupo y si una posición o una camiseta están disponibles.
- Socio decide si puede inscribirse a un equipo (su edad y que no tenga otro equipo en ese deporte).
- Instalacion decide si un horario existe y se puede reservar.
- Turno decide si cumple el mínimo de 2 entrenadores y 1 fisioterapeuta.
- Tienda decide si una venta se puede realizar, y CalculadoraDescuento decide el descuento (D3).

Cuando una regla necesita varios objetos, la coordinadora combina las respuestas y siempre las consulta antes de crear o modificar algo. Eso cumple RNF4 (un rechazo no deja el sistema a medias) y RNF5 (cada rechazo indica la regla incumplida).

Los permisos por rol se resuelven con un menú fijo por tipo de usuario dentro de Consola, que se elige una sola vez después del login. La contraseña la verifica Autentificador.

Costos asumidos:

- Las entidades quedan más relacionadas entre sí.
- Las coordinadoras crecen, sobre todo ClubDeportivo.
- Cada menú ofrece solo lo propio de su rol por construcción, y no porque la lógica lo verifique.

Toda la información se guarda en archivos JSON dentro de una carpeta distinta a la del código fuente. Se eligió JSON porque hay atributos que no aplican a todos los deportes: un campo que no corresponde simplemente no se escribe, así que no hay que representar valores nulos.

Las entidades se referencian entre sí por identificador (por ejemplo, un socio por su id y un equipo por su nombre). Por eso, al abrir la aplicación, los datos se cargan en un orden fijo (usuarios, club deportivo y tienda) y las referencias se enlazan al leer.

Los cambios se guardan **al terminar cada operación** que modifica datos. Si la aplicación se cae, solo se pierde la operación en curso, y el estado en disco siempre es coherente con el de memoria.

Costos asumidos:

- Cada guardado reescribe el archivo completo del componente.
- Hay que depender de una librería JSON.

A continuación, se listan y describen las decisiones más representativas durante el proceso de diseño. Estas fueron elegidas y documentadas dado el nivel de desacuerdo que generaron entre los integrantes del grupo o por su importancia en el resultado final del diseño.

D1: ¿En qué momento se guardan los datos en los archivos de persistencia?

- **Problema:** es necesario definir en qué momento se escriben en disco los cambios hechos en memoria.
- **Alternativas:**
    i. Guardar solo al cerrar la aplicación.
    ii. Guardar tras terminarse cada operación que modifica datos. La consola llama a guardarTodo() después de cada operación que modifica los datos.
    iii. Mixta: guardar tras operaciones críticas (ventas, inscripciones, resultados) y al cerrar la aplicación.
- **Elegida:** la segunda.
- **Ventajas:** si la aplicación se cae, solo se pierde la operación en curso. El estado en disco siempre es coherente con el de memoria.

- **Desventajas:** más escrituras en disco y más trabajo en cada operación. La capa de persistencia debe invocarse desde cada operación que modifica datos.
- **Relación:** RNF2 y la restricción "toda la información debe ser persistente".

D2: ¿Dónde se guardan las reglas que varían por deporte?

- **Problema:** puntos por victoria, empate, mínimo de jugadores y similares cambian según el deporte, y hay que decidir dónde se representan.
- **Alternativas:**
  - Condicionales en la lógica (if deporte == "Futbol"...).
  - Una subclase de Deporte por cada deporte.
  - Atributos de Deporte leídos por la lógica.
- **Elegida:** la tercera.
- **Ventajas:** agregar o ajustar un deporte consiste en crear o editar una instancia de la clase Deporte, sin tocar la lógica de partidos ni competencias.
- **Desventajas:** solo permite cubrir las reglas parametrizables. Las estadísticas propias de cada deporte siguen necesitando una subclase de EstadisticaJugador. Además, los atributos que no aplican a un deporte en específico quedan nulos Por ejemplo, tenis tendría posiciones, puntosVictoria, puntosDerrota, puntosEmpate con valores nulos lo cual obliga a programar defensivamente contra NullPointerException.
- **Relación:** RNF1, RF2, RF3

D3: ¿Cómo modelar a los usuarios que hacen compras en las tiendas del Club?

- **Problema:** Socio y Empleado compran con la misma lógica, pero con distinto descuento (8% con código frente a 15%), y puntosFidelidad estaba duplicado.
- **Alternativas:**
  - Clase abstracta Comprador intermedia.
  - Interface Comprador.
  - Clase Comprador separada.
  - Sin clase nueva, con condicionales por tipo de usuario. CalculadoraDescuento es una clase que Tienda conserva con la lista de códigos válidos.
- **Elegida:** la 4, con puntosFidelidad movido a Usuario.
- **Ventajas:** no cambia la jerarquía del análisis. Venta se asocia a Usuario y sirve a todos los roles, incluido el Administrador. puntosFidelidad queda en un solo lugar.
- **Desventajas:** el descuento depende de ifs anidados (tipo de usuario y código). Un tipo nuevo de descuento obliga a editar ese código, que es justo lo que la alternativa 1 evitaba.
- **Relación:** RF17, RF18, RF20, RF21 y regla de negocio 11.

D4: ¿Quién inscribe a un socio en un equipo o disciplina?

- **Problema:** el enunciado no dice quién debe realizar la inscripción.
- **Alternativas:**
  - El Entrenador inscribe al socio.
  - El Administrador inscribe al socio.
  - El propio Socio se inscribe.
- **Elegida:** la tercera.
- **Ventajas:** menos carga operativa para entrenadores y administrador, y es coherente en equipos y disciplinas individuales.
- **Desventajas:** el entrenador no controla quién entra a su equipo. Por eso el sistema debe validar solo cupo, edad y número de camiseta único, y el socio elige su posición. Aunque no incumple ninguna regla de negocio puede ser problemático desde el punto de vista deportivo.
- **Relación:** RF8, RF9, RNF3 y regla de negocio 1

D5: ¿Cómo se controlan las operaciones permitidas por rol?

- **Problema:** cada operación solo debe estar disponible para su rol (por ejemplo, el Administrador no reserva instalaciones ni se inscribe a equipos), y hay que decidir dónde se controla estas limitaciones de cada usuario.
- **Alternativas:**
  - Condicionales por tipo de usuario repartidos en la lógica (if (usuario instanceof Administrador)).
  - Cada tipo de usuario declara las operaciones que puede ejecutar mediante puede(String operacion), y se consulta tanto para dibujar el menú como dentro de la lógica.
  - El mismo puede(String operacion), pero consultado solo para dibujar el menú.
  - Una clase de menú por rol.
  - Un menú fijo por rol como método dentro de una sola clase ConsolaClub. Se llaman de acuerdo al tipo de usuario tras el login.
- **Elegida:** la 5.
- **Ventajas:**
  - No hay clase ni enum de operaciones, ni nombres de operación como texto.
  - Usuario queda solo con sus datos.
  - El condicional por tipo de usuario aparece en un solo punto, justo después del login.
  - Las acciones comunes se escriben una sola vez.
  - Para saber qué hace un rol basta abrir su menú.
  - La pertenencia (el Socio solo se inscribe a sí mismo) queda garantizada porque el menú le pasa su propio usuario autenticado.
  - La lógica no se protege a sí misma: la restricción de roles se cumple por construcción y se confía en que cada menú ofrece solo lo propio de su rol.

    - ConsolaClub concentra los cuatro menús y la lectura de datos de todas las operaciones, así que puede crecer. Si es necesario se divide por rol sin tocar la lógica.
    - Los permisos quedan repartidos en los menús, no en un solo lugar.
- **Relación:** RNF3, RF8, RF9, RF12 y regla de negocio 10.

D6: Alcance de la restricción de bebidas calientes

- **Problema:** el enunciado prohíbe despachar una bebida caliente a un socio dentro de una cancha de pádel o una mesa de tenis de mesa techada, y también que un socio con bebida caliente use esas instalaciones. El sistema solo conoce reservas, no la ubicación física ni lo que cada socio lleva en la mano.
- **Alternativas:**
    - Implementar las dos direcciones de la regla, lo que exigiría registrar el ingreso a las instalaciones y las bebidas en posesión.
    - Impedir solo la venta de bebidas calientes a un socio con una reserva activa de pádel o de mesa techada.
    - No implementar la restricción.
- **Elegida:** la 2.
- **Ventajas:** usa datos que el sistema ya tiene (la reserva, si la instalación es techada y si el producto es bebida caliente) y no obliga a modelar ingresos ni consumo.
- **Desventajas:**
    - Cubre solo una de las dos direcciones de la regla.
    - Una reserva activa no prueba que el socio esté físicamente dentro: puede rechazar la venta a quien reservó pero aún no llegó, y no cubre a quien entra sin reservar.
    - Cambia las reglas de negocio 6 y 12 del análisis, por lo que debe quedar como registrado como un no-objetivo.
- **Relación:** RF18, No objetivo 8 y reglas de negocio 6 y 12

D7: ¿Cómo se reparten las decisiones de las reglas de negocio?

- **Problema:** muchas reglas dependen de varios objetos (una inscripción mira al Socio y al Equipo, una reserva mira los horarios de la Instalacion, una venta mira inventario) y hay que decidir qué clase las decide.
- **Alternativas:**
    - Estilo centralizado: una sola clase decide todas las reglas.
    - Clases de control separadas (Control*) que deciden cada regla.
    - Principio de Experto en Información: cada regla la decide la clase que tiene los datos necesarios.
- **Elegida:** la 3. Una regla que usa datos de un solo objeto la decide ese objeto (Equipo.tieneCupo, Instalacion.sePuedeCrearReserva, Turno.cumpleFisioterapeutaEntrenadores). Una regla que usa varios la decide el objeto

con la información principal, que consulta a los demás (Socio.sePuedeCrearInscripcionEquipo(equipo) usa sus propias inscripciones y le pregunta a Equipo si tiene cupo). Las coordinadoras combinan las respuestas y siempre las consultan antes de crear algo.

- **Ventajas:**
  - No agrega clases.
  - Las entidades tienen comportamiento propio, no solo getters y setters.
  - La regla vive junto a los datos que usa.
  - Hay menos indirección.
  - Se valida antes de crear, así que un rechazo no deja estados parciales.
- **Desventajas:**
  - Las reglas que cruzan varios objetos quedan repartidas entre ellos, y la coordinadora debe recordar consultarlas todas.
  - Las entidades quedan más acopladas entre sí.
  - Las coordinadoras crecen (ver D9).
  - Aplicar el criterio exige juzgar caso por caso qué objeto tiene la información principal.
- **Relación:** RF2, RF3, RF8, RF9, RF11, RF12, RF14, RF17, RF18, RF21, RNF4 y RNF5.

D8: ¿Hay una clase abstracta para las inscripciones?

- **Problema:** InscripcionEquipo, InscripcionIndividual e InscripcionCompetencia son vínculos distintos, y hay que decidir si comparten una superclase.
- **Alternativas:** clase abstracta Inscripcion; interface Inscripcion; sin superclase.
- **Elegida:** la 3.
- **Ventajas:** cada inscripción guarda solo lo que le corresponde. Los atributos que quedaron no son comunes (cada tipo vincula elementos distintos: socio y equipo, socio y deporte, equipo y competencia). Hay una jerarquía menos que mantener.
- **Desventajas:** no se pueden tratar las tres de forma uniforme, aunque hoy ningún código lo necesita. Si apareciera comportamiento común, habría que introducir la superclase.
- **Relación:** RF3, RF8 y RF9.

D9: ¿Quién recibe las solicitudes de cada componente y guarda las colecciones?

- **Problema:** hace falta una clase que reciba las solicitudes, sea dueña de las colecciones raíz (equipos, competencias, partidos, instalaciones; productos y ventas) y ordene el guardado tras cada operación (D1).
- **Alternativas:**
  - Sin coordinadora: la consola llama directamente a los Control y a las entidades.
  - Colecciones globales o estáticas.
  - Una sola coordinadora para todo el club.

    - Una clase por caso de uso.
    - Dos coordinadoras, una por componente.
- **Elegida:** la 5, con ClubDeportivo (componente deportivo) y Tienda (componente comercial, incluida la cafetería). El nombre de Tienda puede cambiar para que sea más diciente.
- **Ventajas:**
    - Hay un único punto de entrada por componente, que coincide con la partición de componentes.
    - El flujo validar, crear, guardar y persistir vive en un solo lugar.
    - El guardado de D1 queda concentrado en un solo lugar.
    - Las entidades y los Control se mantienen simples.
- **Desventajas:**
    - Riesgo de clase grande, sobre todo en ClubDeportivo, que recibe las operaciones de inscripciones, partidos, reservas, estadísticas, entrenamientos y turnos.
    - Cualquier cambio de flujo la afecta.
    - Tienda también atiende la cafetería, y debe consultar a ClubDeportivo por las reservas activas (RF14).
- **Relación:** RNF2, RNF4, RF17 y RF18

D10: ¿Dónde se calculan los reportes?

- **Problema:** RF7 pide reportes de ventas (por rubro y por periodo) y deportivos (asistencia, resultados y evolución de rankings), que se calculan a partir de datos ya existentes.
- **Alternativas:**
    - Una clase GeneradorReportes para todos.
    - Dos clases, una de reportes de ventas y otra de reportes deportivos, dentro de cada componente.
    - Que cada coordinadora calcule los reportes de su componente.
- **Elegida:** la 3.
- **Ventajas:**
    - No agrega clases.
    - Cada coordinadora ya es dueña de los datos que recorre el reporte.
    - Cada componente conserva sus reportes, sin dependencias cruzadas.
- **Desventajas:**
    - Las coordinadoras crecen más, y mezclan el flujo de operaciones con el de reportes.
    - Un reporte que cruzara ventas y deporte no tendría dónde vivir (hoy no existe ninguno).
- **Relación:** RF7 y regla de negocio 8.

D11: Formato y estructura de los archivos de persistencia

- **Problema:** la información debe guardarse en archivos (restricción técnica). Hay atributos que no aplican a ciertos deportes, registros con subtipos y objetos que se referencian entre sí, y hay que decidir el formato y cuántos archivos se usan.
- **Alternativas:**
  - Texto plano delimitado
  - JSON
  - Serialización binaria de Java.
- **Elegida:** la 2, con un archivo JSON por componente (usuarios, club deportivo y tienda), cada uno con un arreglo por colección y las referencias entre objetos por identificador. Cada archivo tiene su clase (PersistenciaUsuariosJson, PersistenciaClubDeportivoJson y PersistenciaTiendaJson), a las que se accede con CentralPersistencia que entrega cada clase de persistencia.
- **Ventajas:**
  - Un campo que no aplica a un deporte no se escribe, así que no hay que representar nulos en el archivo.
  - Cada subtipo define su constante TIPO y tiene sus propios campos.
  - Los archivos son legibles para preparar datos de prueba.
  - Un archivo por componente refleja la partición del sistema.
- **Desventajas:**
  - Java no incluye librería JSON, así que se depende de una.
  - Las entidades no se pueden serializar directamente por sus referencias circulares: cada cargarX y salvarX convierte a mano con identificadores, y la carga debe seguir un orden.
  - Cada guardado reescribe el archivo completo del componente.
- **Relación:** RNF2, la restricción de persistencia en archivos fuera de la carpeta del código fuente, y D1

Diagramas de Clases

Se escogió dividir el programa en 11 paquetes de acuerdo con los componentes identificados anteriormente y siguiendo las relaciones identificadas en la fase de análisis. Esto permitió la elaboración de un diagrama con mayor claridad y particionado de manera lógica. Se encuentran entregados como archivos PDF dada la necesidad de hacer zoom para apreciar los nombres de los atributos y métodos de las clases distribuidos en los diferentes paquetes.

Nota: Se agregaron métodos y se modificaron los nombres de otros en la última fase de implementación por lo cual estos cambios no alcanzaron a verse reflejados tanto en los diagramas de clases como en los de secuencia. No obstante, el flujo y la lógica general siguen el mismo comportamiento y se acogen a los principios acordados en la toma de decisiones por lo cual la naturaleza del diseño no se ve afectada en sobremanera.

Diagramas de Secuencia

Se escogió diagramar tres operaciones que se consideraron integrales y representativas de la funcionalidad del sistema. El primero se relaciona con la operación deportiva (inscripción de un socio a un equipo), el segundo con la operación comercial (una venta en la tienda deportiva con descuento) y el tercero con el inicio de la aplicación.

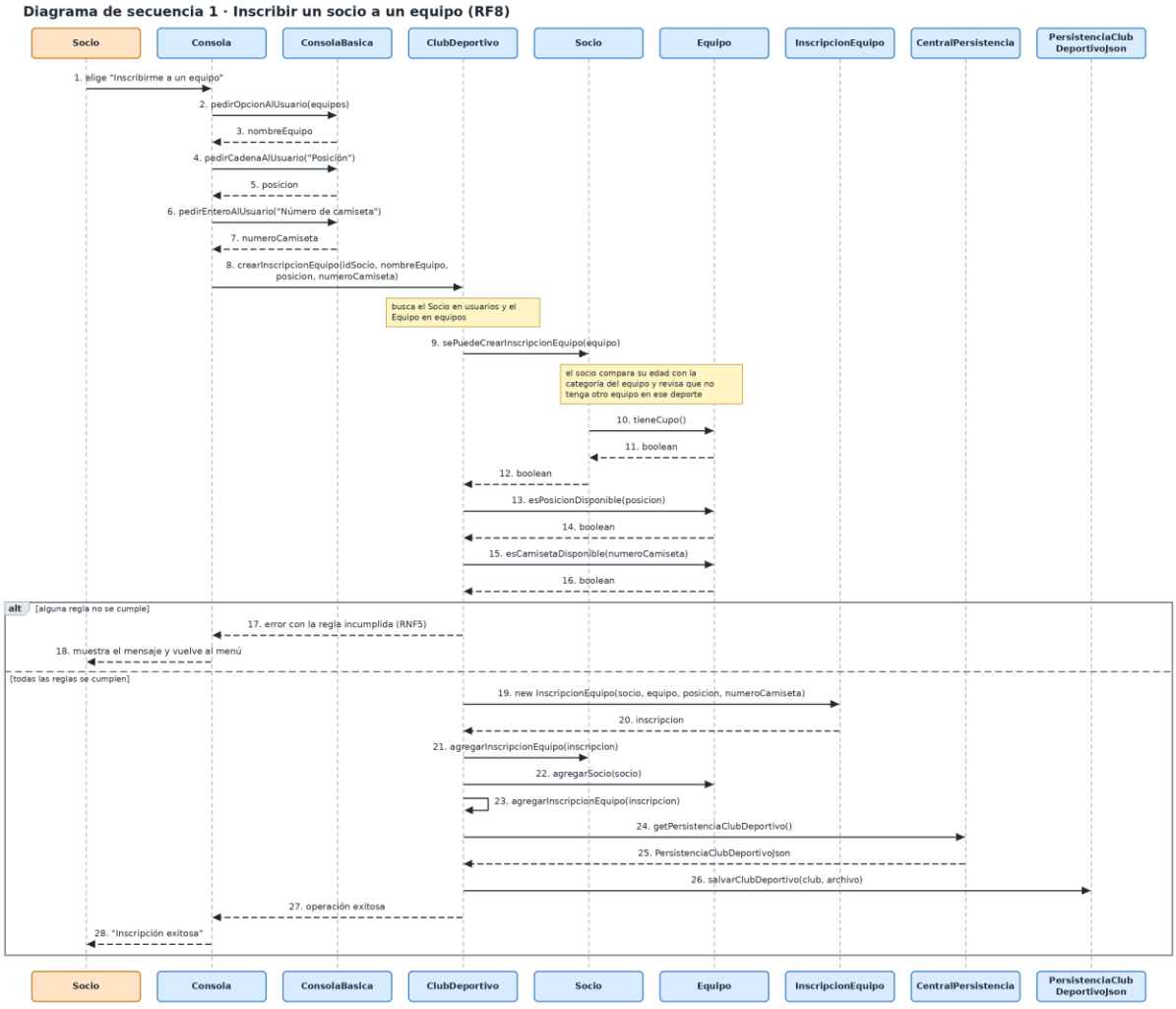
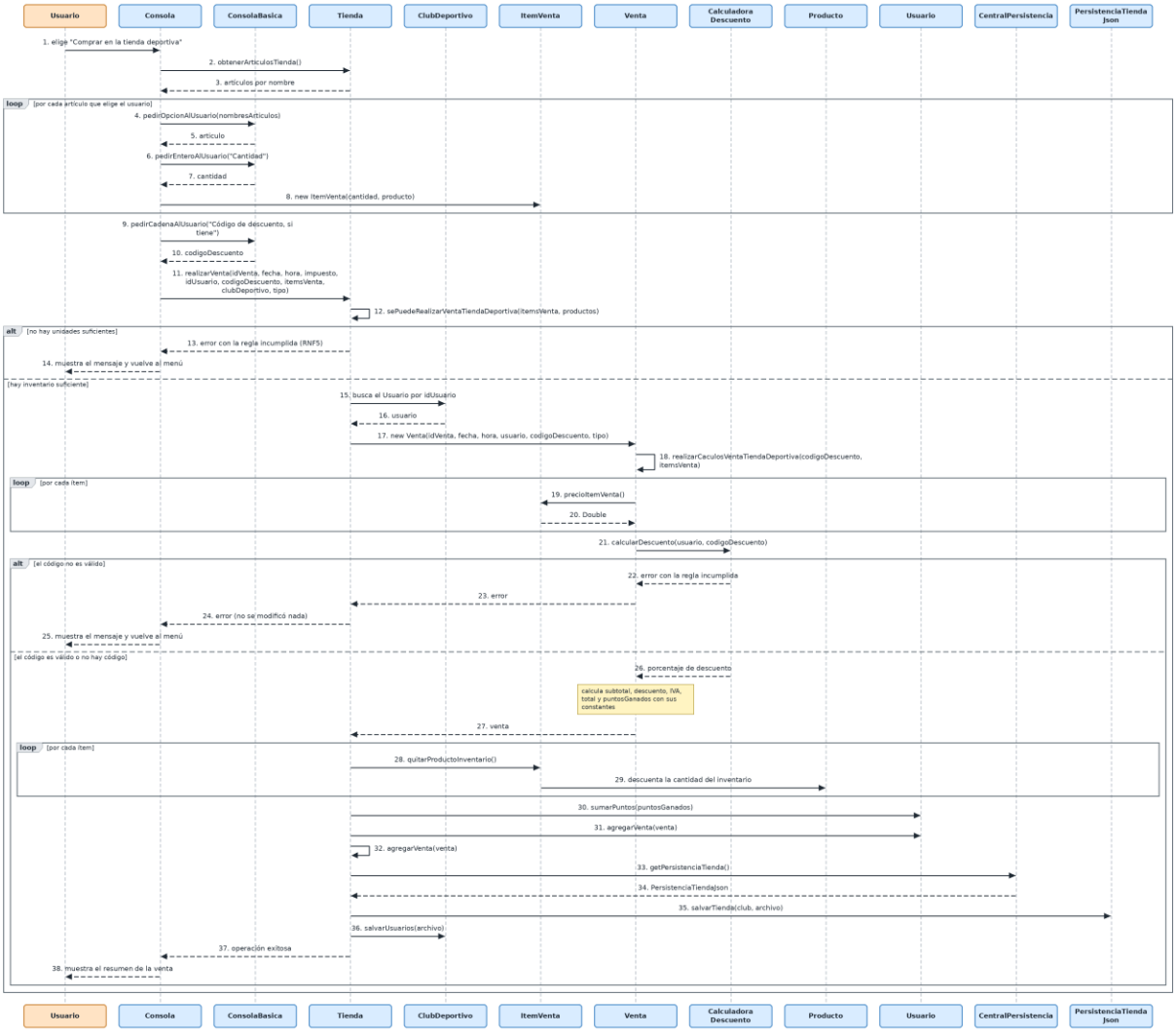
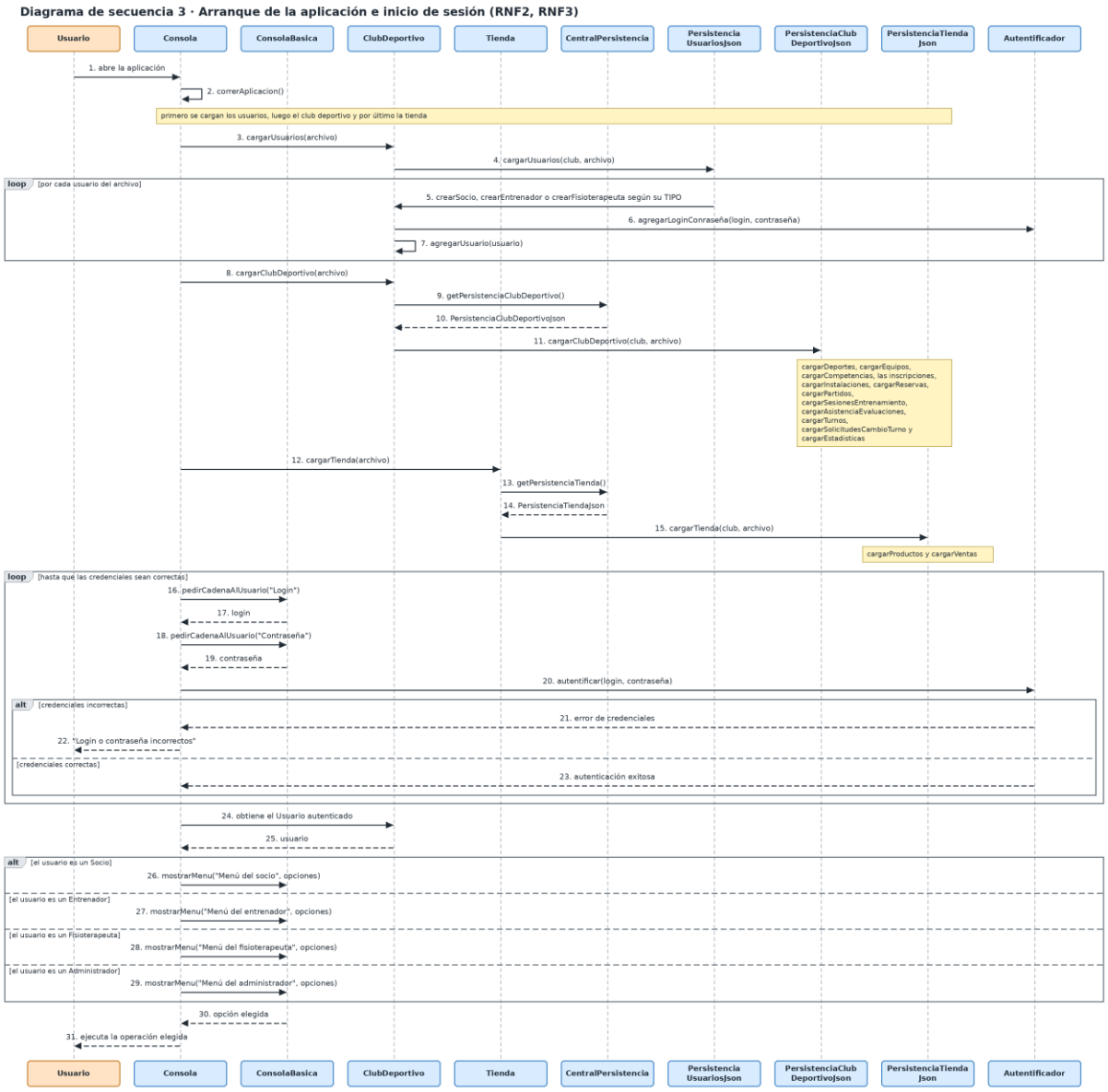


Diagrama de secuencia 2 · Realizar una venta en la tienda deportiva con código de descuento (RF17)





Uso de I.A

Se aprovecharon las capacidades de la I.A para validar y mantener la coherencia del diseño a lo largo de las iteraciones del proceso. En muchas ocasiones, como consecuencia de los cambios de decisiones, existió la necesidad de replantear y editar múltiples secciones en la documentación del diseño. En estos escenarios, la I.A proveyó una forma eficiente de mantener la actualización y coherencia del documento. Asimismo, se usó para encontrar errores y refinar la selección e implementación de métodos, y los artefactos elaborados a partir de estos. Por último, la I.A fue clave en el proceso de poblar los archivos persistentes con datos de mockup, y para hacer el

debugging, resaltando una variedad de errores (de signo, silenciosos e inclusive omisiones completas de métodos y atributos).